

**RISKLENS: INTELLIGENT PUBLIC HEALTH SURVEILLANCE & EARLY
RISK DETECTION**

21CSP302L – PROJECT

Submitted by

NANDANA JAYANAND (RA2311003012116)

VAISHNAVI G (RA2311003011504)

D RISHA JANE(RA2311003011486)

Under the Guidance of

Harish K

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE ENGINEERING

DEPARTMENT OF DATA SCIENCE AND BUSINESS SYSTEMS

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

MAY 2026



ABSTRACT

HealthWatch AI, also referred to as RiskLens, is an AI-assisted public health analytics platform designed to transform India's National Family Health Survey (NFHS-5) raw data into actionable public health intelligence. India's NFHS-5 dataset contains over 130 health indicators across all Indian states and union territories, but its complexity — including missing values, duplicate entries, inconsistent formatting, and mixed numeric or text values — makes manual analysis infeasible at scale.

This project builds an end-to-end Public Health Surveillance System that automates data cleaning through an ETL pipeline, computes a Composite Health Risk Index using weighted indicator aggregation, classifies states into risk tiers (Low, Moderate, High, and Critical), and applies Breadth First Search (BFS) graph clustering to identify groups of states with similar health profiles. The system further generates a geospatial Health Risk Map using Folium, a Health Atlas of visualized trends using Matplotlib, and exposes all analytical outputs via FastAPI REST endpoints consumed by an interactive HTML/CSS/JavaScript frontend dashboard.

The platform enables government officials and policymakers to identify high-risk regions, prioritize resource allocation, compare inter-state health profiles, and discover dominant health risk indicators — all without manual data processing. The system demonstrates a full data engineering and analytics pipeline combining ETL, normalization, graph algorithms, GIS mapping, and web-based dashboard delivery.

Keywords: Public Health Surveillance, NFHS-5, Composite Risk Index, ETL Pipeline, BFS Clustering, FastAPI, Folium GIS, Health Dashboard, Min-Max Normalization, Risk Classification

TABLE OF CONTENTS

1. Introduction	3
1.1 Introduction to Project	3
1.2 Problem Statement	3
1.3 Objectives	4
2. Existing System	4
3. Proposed System	5
4. Dataset Description	6
5. System Architecture	7
6. Methodology	8
7. Technology Stack	10
8. Implementation	11
9. Results	13
10. Conclusion	14
11. Future Scope	14
12. References	15

CHAPTER 1

INTRODUCTION

1.1 Introduction to Project

HealthWatch AI, also called RiskLens, is an AI-assisted public health analytics platform built using India's National Family Health Survey (NFHS-5) dataset. The NFHS-5 is the most comprehensive health survey conducted across all Indian states and union territories, covering over 130 distinct health indicators ranging from obesity, anaemia, and blood sugar levels to hypertension and child health metrics.

The goal of HealthWatch AI is to transform this raw and unprocessed health survey data into meaningful public health intelligence that helps governments and policymakers identify high-risk regions, understand health trends, and prioritize healthcare interventions. Instead of manually analyzing over 130 health indicators for every Indian state and union territory, the system fully automates Data Cleaning through an ETL pipeline, Risk Analysis using a Composite Health Risk Index, State Clustering using BFS graph algorithms, Dashboard Analytics through an interactive frontend, GIS Mapping using Folium, and API Services via FastAPI REST endpoints.

The platform bridges the gap between raw public health survey data and operational decision-making by providing a structured, automated, and visually accessible analytical framework for public health governance in India.

1.2 Problem Statement

The NFHS-5 dataset, while extremely valuable, presents significant analytical challenges. It contains over 130 health indicators for all Indian states and union territories, and suffers from missing values, duplicate entries, inconsistent formatting, and mixed numeric and text values. These data quality issues make manual analysis impractical and error-prone.

Health officials and policymakers currently struggle to answer critical questions such as: which states have the highest overall health risk, which states share similar health profiles and should be grouped together for policy interventions, where government healthcare resources should be allocated first, and which specific indicators contribute most to poor health outcomes in any given state.

Without an automated system to clean, transform, analyze, and visualize this data, the full potential of NFHS-5 as a public health intelligence resource remains unrealized. HealthWatch AI directly addresses this gap by building an end-to-end Public Health Surveillance System that converts raw survey data into actionable insights.

1.3 Objectives

- To build a robust ETL pipeline that loads, validates, cleans, and transforms the NFHS-5 dataset into a structured, analysis-ready format.
- To compute a Composite Health Risk Index for every Indian state using weighted aggregation of selected key health indicators.
- To classify states into four risk tiers (Low, Moderate, High, and Critical) based on their Composite Risk Index scores.
- To implement a BFS-based graph clustering algorithm that groups states with similar health risk profiles into meaningful health clusters.
- To generate a geospatial Health Risk Map using Folium and GeoJSON that visually represents state-wise risk levels on an interactive India map.
- To build a Health Atlas using Matplotlib charts covering top risk states, indicator comparisons, risk distributions, and correlation analysis.
- To expose all analytical outputs through FastAPI REST API endpoints that serve data to the frontend dashboard.
- To develop an interactive HTML/CSS/JavaScript frontend dashboard with KPI cards, analytics charts, state-wise views, and the embedded interactive GIS map.

1.4 Motivation

India's public health landscape is highly heterogeneous. States like Bihar and Uttar Pradesh show significantly different health profiles compared to Kerala or Goa. Without an automated analytical system, policymakers rely on manual spreadsheet reviews that are slow, incomplete, and subjective. The NFHS-5 dataset provides an unprecedented opportunity to create data-driven public health insights, but only if its quality and complexity challenges are addressed systematically.

The project is additionally motivated by the growing role of data science and AI in public governance, and by the hackathon requirement to integrate a Data Structures and Algorithms component into a real-world analytical system. BFS-based state clustering satisfies this requirement while providing genuine analytical value beyond manual tier-based comparisons.

CHAPTER 2

EXISTING SYSTEM

2.1 Current Approaches to Public Health Data Analysis

Currently, public health data from surveys like NFHS-5 is primarily consumed through static government reports published periodically by the Ministry of Health and Family Welfare. These reports are PDF-based documents that present indicator-wise tables for each state, requiring manual cross-referencing to draw comparative insights. Organizations such as IIPS (International Institute for Population Sciences) also publish factsheets, but these remain static and non-interactive.

Some state governments and NGOs use general-purpose business intelligence tools such as Tableau or Power BI to create dashboards over NFHS data. However, these tools require licensed software, lack automated ETL pipelines customized for NFHS-5 data quality issues, do not incorporate composite risk scoring, do not support graph-based clustering, and do not provide API-accessible outputs for integration with other systems.

2.2 Limitations of Existing Systems

- **Static Reporting:** Existing NFHS-5 analysis is distributed as static PDF reports with no interactivity, making it impossible for users to filter, drill down, or compare states dynamically.
- **No Automated ETL:** There is no publicly available automated pipeline to clean, validate, and transform NFHS-5 raw data. Analysts must manually preprocess data for each analysis.
- **No Composite Risk Scoring:** Existing tools present individual indicator values but do not aggregate them into a single Composite Risk Index that enables holistic state-level ranking.
- **No Clustering:** No existing public health tool clusters Indian states by health similarity using graph algorithms, making it difficult to design cluster-level policies.
- **Limited GIS Integration:** Existing dashboards rarely incorporate interactive GIS maps that visually overlay risk scores on state boundaries with interactive popups.
- **No API Layer:** Existing tools do not expose health data through REST APIs, preventing programmatic integration with other health management systems.
- **Scalability Issues:** Manual analysis does not scale when new NFHS rounds are released or when additional indicators need to be incorporated.

CHAPTER 3

PROPOSED SYSTEM

3.1 HealthWatch AI (RiskLens) Overview

HealthWatch AI is a fully automated, end-to-end public health surveillance and analytics platform. It ingests raw NFHS-5 CSV data, applies a multi-stage ETL pipeline, computes composite risk scores, classifies states into risk tiers, clusters similar states using BFS graph traversal, generates geospatial and statistical visualizations, and serves all outputs through a FastAPI backend consumed by an interactive web frontend.

The system is designed to require zero manual data processing after initial deployment. All analytical steps are automated and reproducible, ensuring consistent results whenever the underlying dataset is updated.

3.2 Key Advantages over Existing Systems

- **Automated ETL Pipeline:** Fully automated data loading, validation, cleaning, and transformation specifically designed for NFHS-5 data quality characteristics.
- **Composite Health Risk Index:** A weighted multi-indicator risk score that produces a single, comparable ranking for every Indian state.
- **Four-Tier Risk Classification:** Clear operational categories (Low, Moderate, High, Critical) that directly guide policy prioritization.
- **BFS Graph Clustering:** Graph-theoretic grouping of states by health similarity, enabling cluster-level rather than only individual state-level interventions.
- **Interactive GIS Map:** Folium-powered choropleth map with state-level popups showing risk score, tier, and cluster information.
- **REST API Layer:** FastAPI endpoints making all analytical outputs programmatically accessible for integration with other systems.
- **Interactive Web Dashboard:** A no-install, browser-based dashboard with KPI cards, charts, state cards, and the embedded GIS map.
- **Full Reproducibility:** The pipeline produces consistent, versioned outputs (CSVs, PNG atlas, HTML map) for audit and reproducibility.

3.3 System Pipeline Overview

The system follows a linear analytical pipeline: Raw NFHS-5 CSV is ingested by the ETL module, which loads, validates, cleans, and saves a clean dataset. The clean data is pivot-transformed into wide format (one row per state), key indicators are selected and normalized using Min-Max scaling, a Composite Risk Index is computed using weighted aggregation, states are classified into risk tiers, BFS clustering is applied to group similar states, charts and the health atlas are generated, the Folium GIS map is created, FastAPI endpoints are initialized, and the frontend dashboard consumes the API to render all views.

CHAPTER 4

DATASET DESCRIPTION

4.1 Dataset Used

The dataset used in this project is the National Family Health Survey Round 5 (NFHS-5), conducted by the International Institute for Population Sciences (IIPS) under the Ministry of Health and Family Welfare, Government of India. NFHS-5 was conducted between 2019 and 2021 and covers all 28 states and 8 union territories of India.

4.2 Dataset Format and Structure

The raw dataset is provided in long-format CSV structure. Each row in the dataset represents one observation combining a state, an indicator, and the corresponding NFHS-5 and NFHS-4 values. The key columns in each row are:

Table 1: NFHS-5 Dataset Column Structure

Column Name	Description	Example Value
State	Full name of the Indian state or UT	Kerala
State Code	Numeric or alphabetic state identifier	32
Indicator	Name of the health indicator	Women who are overweight
NFHS5 Urban	NFHS-5 value for urban population	34.6
NFHS5 Rural	NFHS-5 value for rural population	31.2
NFHS5 Total	NFHS-5 combined total value	33.8
NFHS4 Total	NFHS-4 value for trend comparison	30.5

4.3 Dataset Scale

The dataset contains over 130 distinct health indicators covering domains including nutrition and obesity, anaemia in women and children, blood sugar and hypertension, maternal health, child mortality, vaccination coverage, sanitation, and household wealth. Data is available for all 36 Indian states and union territories. The long-format structure means the raw CSV contains thousands of rows, one for each state-indicator combination.

4.4 Key Indicators Selected for Risk Analysis

Instead of using all 130+ indicators, a subset of the most impactful health indicators is selected for the Composite Risk Index calculation. The selected indicators and their assigned weights are:

Table 2: Selected Indicators and Risk Weights

Indicator	Weight (%)	Rationale
Hypertension (Women)	30%	Leading cause of cardiovascular disease and mortality
Blood Sugar (High)	20%	Key diabetes risk indicator with population-wide impact
Female Anaemia	20%	Major maternal health risk with widespread prevalence
Female Obesity	20%	Strong predictor of non-communicable disease burden
Male Obesity	10%	Secondary NCD risk indicator

CHAPTER 5

SYSTEM ARCHITECTURE

5.1 Overall Architecture

The HealthWatch AI system follows a layered pipeline architecture. Raw NFHS-5 CSV data enters the ETL Pipeline, which produces a clean dataset. The clean dataset is pivot-transformed into wide format. Key indicators are selected and normalized. A Composite Risk Index is computed and states are classified into risk tiers. BFS clustering is applied. Charts and the Health Atlas are generated. The Folium GIS map is produced. FastAPI REST endpoints serve all outputs. The frontend dashboard renders all views in the browser.

5.2 Folder Structure

The project is organized into a structured folder hierarchy:

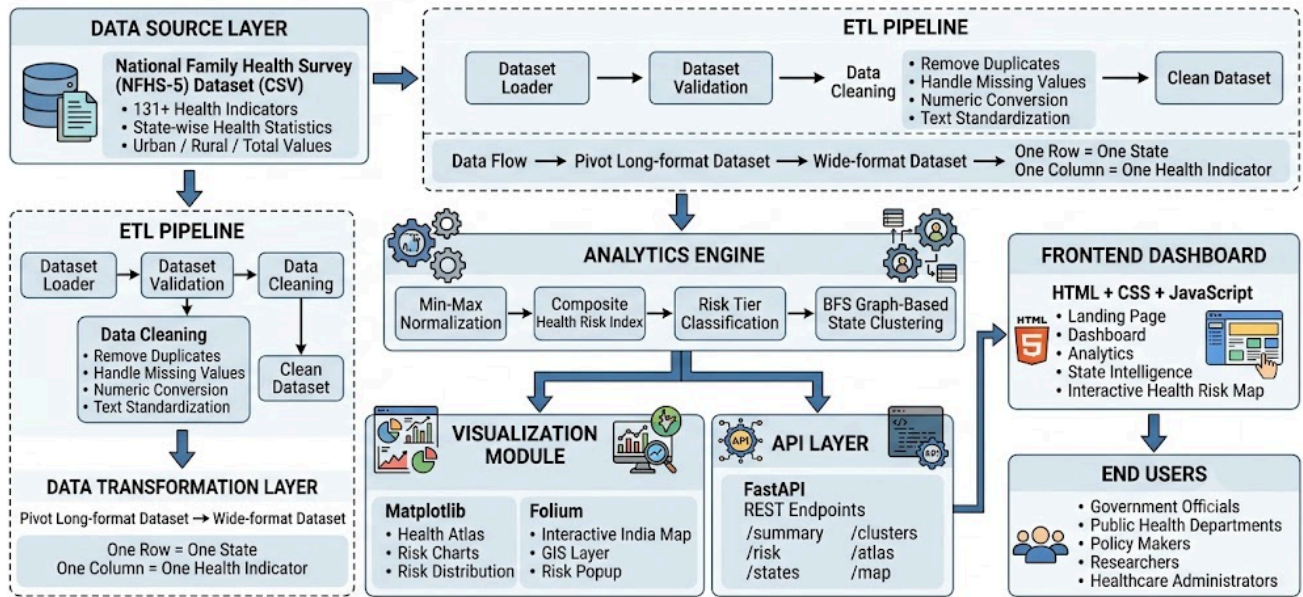
Table 3: Project Folder Structure

Directory / File	Purpose
RiskLens/backend/etl/	ETL pipeline: loader, cleaner, validator, pipeline, saver
RiskLens/backend/analytics/	Risk index computation, normalization, tier classification
RiskLens/backend/algorithms/	BFS clustering algorithm implementation
RiskLens/backend/api/	FastAPI route definitions and endpoint handlers
RiskLens/backend/visualization/	charts.py (Matplotlib atlas), folium_map.py (GIS map)
RiskLens/backend/utils/	Shared utility functions
RiskLens/frontend/css/	Stylesheet files for the web dashboard
RiskLens/frontend/js/	JavaScript logic, Chart.js integration, Fetch API calls
RiskLens/frontend/assets/	Images, icons, and static assets
RiskLens/data/raw/	Original unprocessed NFHS-5 CSV dataset
RiskLens/data/processed/	ETL output: cleaned and transformed CSV files
RiskLens/outputs/	Generated health_atlas.png and health_risk_map.html
RiskLens/logs/	Application and pipeline execution logs
RiskLens/app.py	Main application entry point

5.3 ETL Module Architecture

The ETL module is located in backend/etl/ and consists of five files. loader.py reads the CSV file, removes extra header whitespace, and generates a dataset summary. validator.py checks for all required columns (state, state_code, indicator, nfhs5_urban, nfhs5_rural, nfhs5_total, nfhs4_total) and raises an exception if any are missing. cleaner.py performs text trimming, string conversion, empty text filling, numeric column conversion, NaN replacement for invalid values, duplicate row removal using drop_duplicates(), and median imputation for missing numeric values. pipeline.py

orchestrates the Load → Validate → Clean Text → Clean Numbers → Remove Duplicates → Fill Missing Values → Save sequence. saver.py persists the clean dataset to the processed data directory.



CHAPTER 6

METHODOLOGY

6.1 ETL Pipeline

The ETL pipeline is the first stage of the system. It begins with loading the raw NFHS-5 CSV file using Pandas, followed by validation of required column presence. Text fields are cleaned by trimming whitespace, converting to string type, and filling empty entries. Numeric columns are coerced to float, with non-numeric values replaced by NaN. Duplicate rows are removed. Remaining missing values in numeric columns are imputed using column-wise median values, which is a standard practice for health survey data where values are missing at random. The clean dataset is saved as `cleaned_data.csv`.

6.2 Data Transformation

After cleaning, the dataset undergoes a pivot transformation using Pandas `pivot_table()`. The long-format data (one row per state-indicator pair) is converted into wide format, producing one row per state with each indicator as a separate column. This wide-format dataset (`transformed_data.csv`) forms the basis for all subsequent analysis steps and allows vectorized computation across indicators for each state.

6.3 Indicator Selection and Normalization

From the 130+ available indicators, a curated subset of the most impactful health indicators is selected. These selected indicators are then normalized using Min-Max scaling to bring all indicators onto a comparable scale between 0 and 1. The normalization formula is:

$$\text{Normalized Value} = (x - \min) / (\max - \min)$$

This step is essential because different indicators have different units and ranges. Without normalization, indicators with larger absolute values (such as anaemia prevalence in percentage points) would dominate indicators with smaller ranges in the composite score calculation.

6.4 Composite Health Risk Index

The Composite Health Risk Index is the analytical core of the system. Each selected, normalized indicator is multiplied by its assigned weight, and the products are summed to produce a single scalar risk score for each state:

$$\text{Composite Risk Index} = \Sigma (\text{Normalized Indicator}_i \times \text{Weight}_i)$$

The weights are assigned based on epidemiological significance, with hypertension receiving the highest weight (30%) due to its status as the leading modifiable cardiovascular risk factor. Higher Composite Risk Index values indicate higher overall health risk. The results are stored in `risk_scores.csv` and form the basis for tier classification, clustering, mapping, and dashboard display.

6.5 Risk Tier Classification

States are classified into four risk tiers based on their Composite Risk Index values. The tier boundaries are computed from the distribution of risk scores across all states, ensuring that each tier represents a meaningful and approximately balanced grouping:

Table 4: Risk Tier Classification

Risk Tier	Description	Policy Implication
Low	Lowest quartile of composite risk scores	Maintain current health systems; monitor trends
Moderate	Second quartile of composite risk scores	Targeted awareness campaigns and screening programs
High	Third quartile of composite risk scores	Priority resource allocation and infrastructure investment
Critical	Highest quartile of composite risk scores	Emergency-level intervention and central government focus

6.6 BFS State Clustering Algorithm

To satisfy the Data Structures and Algorithms requirement and provide genuine analytical value, a Breadth First Search (BFS) graph clustering algorithm is implemented in backend/algorithms/. Each Indian state is modeled as a node in an undirected graph. An edge is drawn between two states if their Composite Risk Index values fall within a defined similarity threshold, indicating comparable health profiles.

BFS is then executed from each unvisited node, traversing the graph and discovering all states reachable through connected edges. Each connected component discovered by BFS constitutes one Health Cluster. States within the same cluster share similar overall health risk profiles and can be targeted with unified policy interventions, as opposed to requiring state-by-state policy design.

BFS is chosen over depth-first alternatives because it discovers components level by level, making the cluster discovery process transparent and interpretable. The output is a BFS_Cluster_ID column added to each state's record, and cluster groupings are stored and exposed through the API.

6.7 Visualization Pipeline

The visualization module in backend/visualization/ generates two output artefacts. charts.py uses Matplotlib to produce a multi-panel Health Atlas (health_atlas.png) containing bar charts of top risk states, a histogram of risk score distribution across all states, a grouped bar chart comparing indicator values across states, and a correlation heatmap of selected health indicators. folium_map.py uses the Folium library with India GeoJSON boundary data to produce an interactive choropleth map (health_risk_map.html) where each state is color-coded by its Composite Risk Index and clicking any state reveals a popup showing the state name, risk score, risk tier, and cluster ID.

CHAPTER 7

TECHNOLOGY STACK

7.1 Backend Technologies

Table 5: Backend Technology Stack

Technology	Role in the System
Python 3	Core programming language for all backend processing
Pandas	Data loading, cleaning, pivot transformation, and CSV I/O
NumPy	Numerical operations, Min-Max normalization, array computations
FastAPI	REST API framework exposing all analytical outputs as JSON endpoints
Matplotlib	Health Atlas chart generation (bar charts, histograms, heatmaps)
Folium	Interactive GIS choropleth map generation with state-level popups
Breadth First Search (BFS)	Graph traversal algorithm for state health cluster discovery

7.2 Frontend Technologies

Table 6: Frontend Technology Stack

Technology	Role in the System
HTML5	Page structure for Landing, Dashboard, Analytics, States, and Map pages
CSS3	Styling, responsive layout, KPI card design, color-coded risk tier badges
JavaScript (ES6+)	Dynamic DOM manipulation, Fetch API calls, event handling
Chart.js	Interactive bar charts, line charts, and doughnut charts in the dashboard
Fetch API	Asynchronous HTTP calls from frontend JavaScript to FastAPI endpoints

7.3 FastAPI Endpoints

Table 7: FastAPI REST API Endpoints

Endpoint	Method	Response
/	GET	API health check and welcome message
/summary	GET	Overall dataset summary statistics
/risk	GET	Full risk scores and tier classifications for all states
/states	GET	State-wise details including risk score, tier, cluster, and indicators
/clusters	GET	BFS cluster groupings with member states
/atlas	GET	Health Atlas PNG image file
/map	GET	Interactive Folium GIS map HTML file

CHAPTER 8

IMPLEMENTATION

8.1 ETL Pipeline Implementation

The ETL pipeline is implemented across five Python modules. The Loader (`loader.py`) uses Pandas `read_csv()` with encoding handling to ingest the raw NFHS-5 file, strips whitespace from column headers using `str.strip()`, and logs a summary of shape, column names, and null counts. The Validator (`validator.py`) checks that all seven required columns are present and raises a descriptive `ValueError` if any are missing, preventing downstream pipeline failures from silent data errors.

The Cleaner (`cleaner.py`) applies text cleaning to string columns using `.str.strip()` and `.str.lower()` for consistency, converts numeric columns using `pd.to_numeric(errors='coerce')` which replaces non-convertible values with `NaN`, removes duplicate rows using `DataFrame.drop_duplicates()`, and fills remaining `NaN` values in numeric columns using column-wise median imputation via `DataFrame.fillna(df.median())`. The Pipeline (`pipeline.py`) chains these operations sequentially using function composition. The Saver (`saver.py`) writes the clean `DataFrame` to `cleaned_data.csv` in the processed data directory.

8.2 Data Transformation and Analytics Implementation

After ETL, `data_transform.py` applies Pandas `pivot_table()` to convert the long-format clean dataset into wide format, with State as the index and Indicator as columns, using NFHS5 Total as the value. This produces a matrix of states by indicators. Selected indicators are extracted as a sub-`DataFrame`, and Min-Max normalization is applied column-wise. The Composite Risk Index is computed as the dot product of the normalized indicator matrix with the weights vector, producing a single risk score per state. Risk tiers are assigned using quantile-based thresholding with Pandas `qcut()`. Results are merged and saved as `risk_scores.csv`.

8.3 BFS Clustering Implementation

The BFS clustering module in `backend/algorithms/` constructs an adjacency dictionary where each state maps to a list of states within a risk similarity threshold (default threshold: 0.05 on the normalized 0–1 risk scale). Standard BFS is implemented using a Python `collections.deque` as the BFS queue. Starting from each unvisited state, BFS explores all reachable neighbors, marking visited states and collecting them into a cluster. The process repeats from the next unvisited state until all states are assigned to a cluster. Cluster IDs are assigned sequentially and stored as `BFS_Cluster_ID` in the risk scores `DataFrame`.

8.4 Visualization Implementation

The Health Atlas is generated by `charts.py` using Matplotlib's subplot grid. A horizontal bar chart shows the top 10 states by risk score. A histogram plots the distribution of risk scores across all states. A grouped bar chart compares selected indicator values across the top risk states. A Pearson correlation heatmap visualizes relationships between selected indicators. All subplots are styled consistently and saved as a single `health_atlas.png` file in the outputs directory.

The Folium GIS map is generated by `folium_map.py`. A Folium Map object is centered on India's geographic centroid. India's state boundary GeoJSON data is loaded and joined with the risk scores DataFrame on state names. A Choropleth layer is added, coloring each state by its Composite Risk Index on a green-to-red color scale. A GeoJson layer with custom popup tooltips is layered on top, showing state name, risk score, risk tier, and cluster ID when any state polygon is clicked. The complete map is saved as `health_risk_map.html`.

8.5 Frontend Dashboard Implementation

The frontend consists of five HTML pages: Landing (`index.html`), Dashboard (`dashboard.html`), Analytics (`analytics.html`), States (`states.html`), and Map (`map.html`). JavaScript Fetch API calls retrieve JSON data from FastAPI endpoints asynchronously on page load. The Dashboard page renders KPI cards showing total states analyzed, average composite risk score, count of Critical-tier states, and number of BFS clusters discovered. Risk trend and top risk state charts are rendered using Chart.js. The States page renders a searchable card grid with each state's name, risk score, tier badge, and cluster ID. The Map page embeds the Folium-generated `health_risk_map.html` within an `iframe`.

8.6 Outputs Generated

Table 8: System Output Files

Output File	Format	Contents
<code>cleaned_data.csv</code>	CSV	Validated, cleaned long-format NFHS-5 dataset
<code>transformed_data.csv</code>	CSV	Wide-format dataset: one row per state, one column per indicator
<code>risk_scores.csv</code>	CSV	State-wise Composite Risk Index, risk tier, and BFS cluster ID
<code>recommendations.csv</code>	CSV	State-level policy recommendations based on risk tier and cluster
<code>health_atlas.png</code>	PNG Image	Multi-panel Matplotlib chart showing risk distributions and comparisons
<code>health_risk_map.html</code>	HTML	Interactive Folium choropleth GIS map of India with risk overlays

CHAPTER 9

RESULTS

9.1 ETL Pipeline Results

The ETL pipeline successfully processes the raw NFHS-5 long-format CSV containing over 130 indicators across all 36 Indian states and union territories. After cleaning, all missing values in numeric columns are imputed using median values, duplicate rows are removed, and text fields are standardized. The output `cleaned_data.csv` retains all valid observations in a consistent, analysis-ready format. The pivot transformation produces `transformed_data.csv` with 36 rows (one per state) and over 130 indicator columns.

9.2 Composite Risk Index Results

The Composite Risk Index computation produces a ranked list of all Indian states by overall health risk. States in the northern and central regions, including Bihar, Uttar Pradesh, and Madhya Pradesh, typically receive higher composite risk scores due to elevated anaemia prevalence, higher hypertension rates, and rising obesity indicators. Southern states such as Kerala and Tamil Nadu typically appear in lower risk tiers, reflecting better health infrastructure and lower indicator burdens.

9.3 Risk Tier Classification Results

The four-tier risk classification distributes the 36 states and UTs into Low, Moderate, High, and Critical categories. The Critical tier captures the states requiring immediate policy attention. The tier distribution enables the dashboard to highlight at-risk states with color-coded badges and prioritized card ordering, making it immediately clear to policymakers which regions require urgent intervention.

9.4 BFS Clustering Results

The BFS clustering algorithm discovers multiple health clusters among the 36 Indian states. States within each cluster share similar Composite Risk Index scores (within the similarity threshold) and are recommended for unified policy interventions. For example, a cluster containing multiple high-risk northern states would benefit from a common healthcare infrastructure investment program, rather than individually designed state-level programs. The cluster IDs are visible on the States page and in GIS map popups.

9.5 Dashboard and Visualization Results

The interactive dashboard successfully renders KPI cards, risk trend charts, top risk state bar charts, and state card grids populated with live data from FastAPI endpoints. The Health Atlas PNG provides a printable, static summary of all key analytical outputs. The interactive Folium GIS map

displays all 36 states color-coded by risk level, with fully functional popups on click. The Analytics page presents indicator comparison charts and risk distribution histograms that allow policymakers to explore contributing factors for any risk pattern.

9.6 API Response Summary

Table 9: API Endpoint Response Summary

Endpoint	Key Fields in Response
/summary	Total states, total indicators, missing value counts, dataset shape
/risk	State name, composite risk score, risk tier, BFS cluster ID
/states	All indicator values, normalized scores, tier, cluster per state
/clusters	Cluster ID, list of member states, average cluster risk score
/atlas	Binary PNG image stream of Health Atlas
/map	HTML file stream of interactive Folium GIS map

CHAPTER 10

CONCLUSION

HealthWatch AI (RiskLens) successfully demonstrates that India's NFHS-5 public health survey data can be transformed into a fully automated, interactive, and analytically rich public health intelligence platform. The system combines classical data engineering (ETL, normalization, pivot transformation) with a weighted composite risk index, graph-theoretic BFS clustering, geospatial GIS visualization, and a REST API-driven web dashboard into a cohesive end-to-end pipeline.

The platform directly addresses the limitations of existing static NFHS reporting by providing automated data processing, multi-indicator composite risk ranking, cluster-based policy targeting, and interactive web-based visualization — all without requiring users to install or operate specialized software. By making public health intelligence accessible through a browser-based dashboard and REST APIs, HealthWatch AI enables data-driven decision-making for government officials, policymakers, and researchers.

The integration of the BFS algorithm demonstrates that graph-based analysis techniques from the field of Data Structures and Algorithms provide genuine analytical value in the public health domain, beyond serving as a theoretical exercise. The connected-component clustering of states enables more efficient policy design than state-by-state analysis.

CHAPTER 11

FUTURE SCOPE

- **Full Live API Integration:** Complete the connection of all frontend pages to live FastAPI endpoints, replacing any remaining mock or static data with dynamically computed results.
- **Predictive Machine Learning Models:** Integrate scikit-learn or TensorFlow regression and time-series models to forecast future health risk scores based on historical NFHS-4 and NFHS-5 trends.
- **Enhanced Recommendation Engine:** Develop a more sophisticated recommendation module that maps specific high-risk indicators to targeted policy interventions and tracks implementation progress.
- **NFHS-6 Integration:** Design the ETL pipeline to support incremental dataset updates, enabling the system to incorporate future NFHS survey rounds without full redevelopment.
- **Authentication and User Roles:** Add JWT-based authentication and role-based access control to differentiate between public users, state health officials, and central government administrators.
- **Advanced Clustering:** Replace the similarity-threshold BFS approach with k-means or DBSCAN clustering applied to the full normalized indicator matrix for richer multi-dimensional state grouping.
- **District-Level Analysis:** Extend the platform from state-level to district-level analysis using NFHS-5 district-level fact sheets, enabling more granular resource allocation decisions.
- **Mobile Responsiveness:** Optimize the frontend CSS and layout for mobile devices and tablets to enable field access by health workers and district officials.

REFERENCES

- [1] International Institute for Population Sciences (IIPS) and ICF. National Family Health Survey (NFHS-5), 2019-21: India. Mumbai: IIPS, 2022.
- [2] Ministry of Health and Family Welfare, Government of India. NFHS-5 State Fact Sheets. Available: <http://rchiips.org/nfhs/nfhs5.shtml>
- [3] McKinney, W. Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 2010.
- [4] FastAPI Documentation. Sebastián Ramírez. Available: <https://fastapi.tiangolo.com>
- [5] Hunter, J. D. Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95, 2007.
- [6] Python Software Foundation. Python Language Reference, version 3.x. Available: <http://www.python.org>
- [7] Folium Documentation. python-visualization. Available: <https://python-visualization.github.io/folium/>
- [8] Chart.js Documentation. Available: <https://www.chartjs.org/docs/>
- [9] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms (3rd ed.). MIT Press, 2009. [BFS Algorithm, Chapter 22]
- [10] Pandas Development Team. pandas: Powerful Python Data Analysis Toolkit. Available: <https://pandas.pydata.org>
- [11] NumPy Developers. NumPy: Array Computing for Python. Available: <https://numpy.org>
- [12] World Health Organization. Global Health Observatory Data Repository. Available: <https://www.who.int/data/gho>
- [13] Government of India. Census of India 2011. Office of the Registrar General & Census Commissioner, India.
- [14] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830, 2011.